

Java Backend Architecture

Interview Series

Chapter 8.6

Spring Security Architecture

50+ Interview Q&A

Code Examples

Architecture Diagrams

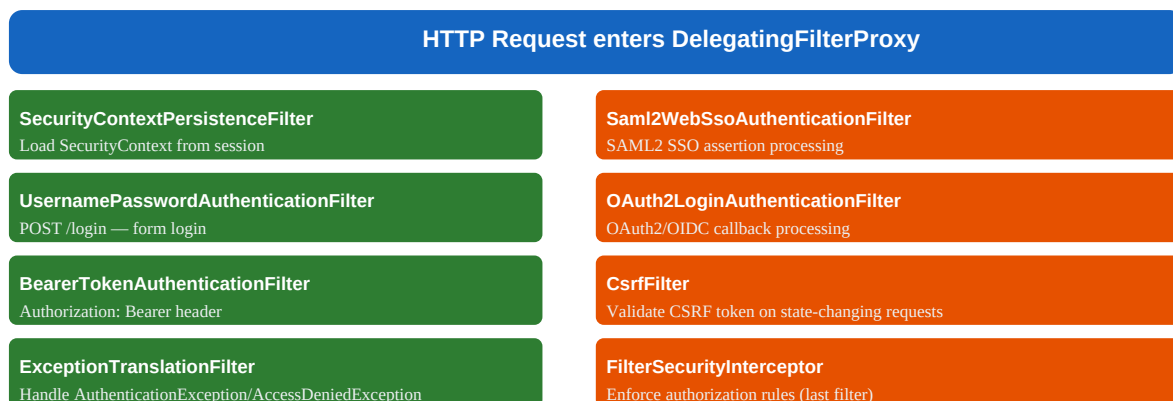
Advanced Topics

Chapter 8.6 — Spring Security Architecture

Introduction

Spring Security is the de facto standard for securing Spring applications. Its architecture is built around a chain of servlet filters that intercept HTTP requests, a pluggable authentication framework, and a flexible authorization model. Understanding the filter chain, SecurityContext, and authentication flow is essential for diagnosing security issues and customizing behavior.

SecurityFilterChain & DelegatingFilterProxy

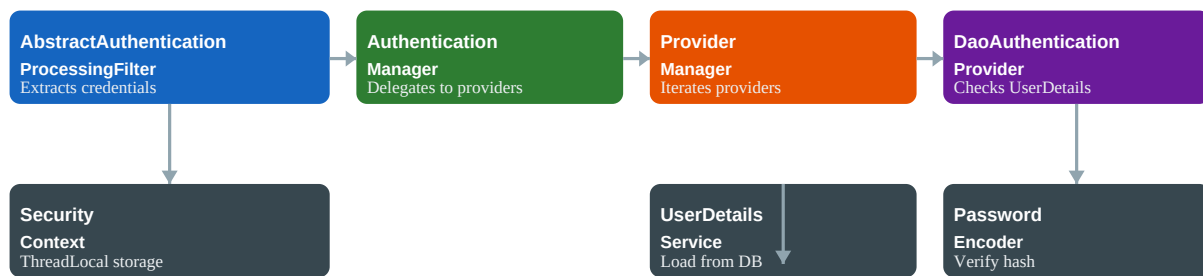


Filters are ordered. FilterChainProxy delegates to the matching SecurityFilterChain. Multiple chains supported.

DelegatingFilterProxy is a standard servlet Filter registered in the servlet container. It delegates to a Spring-managed bean (FilterChainProxy). **FilterChainProxy** contains multiple SecurityFilterChain instances and routes each request to the first matching chain based on the request path pattern. Each chain contains an ordered list of security filters.

```
// Multiple SecurityFilterChain example @Configuration @EnableWebSecurity public class
MultiChainConfig { // Chain 1: API endpoints — stateless JWT @Bean @Order(1) public
SecurityFilterChain apiChain(HttpSecurity http) throws Exception { http
.securityMatcher("/api/**") .csrf(csrf -> csrf.disable()) .sessionManagement(s ->
s.sessionCreationPolicy(STATELESS)) .authorizeHttpRequests(auth -> auth
.requestMatchers("/api/public/**").permitAll() .anyRequest().authenticated())
.oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults())); return http.build(); }
// Chain 2: Web UI — session-based form login @Bean @Order(2) public SecurityFilterChain
webChain(HttpSecurity http) throws Exception { http .authorizeHttpRequests(auth -> auth
.requestMatchers("/login", "/css/**").permitAll() .anyRequest().authenticated())
.formLogin(form -> form.loginPage("/login").defaultSuccessUrl("/home"))
.logout(Customizer.withDefaults()); return http.build(); } }
```

Authentication Flow



After successful auth: Authentication stored in SecurityContextHolder (ThreadLocal). Cleared in SecurityContextPersistenceFilter after response.

```
// Custom UserDetailsService @Service public class CustomUserDetailsService implements
UserDetailsService { private final UserRepository userRepository; @Override public UserDetails
loadUserByUsername(String username) throws UsernameNotFoundException { User user =
userRepository.findByUsername(username) .orElseThrow(() -> new UsernameNotFoundException("User
not found: " + username)); return org.springframework.security.core.userdetails.User
.withUsername(user.getUsername()) .password(user.getPasswordHash()) // Must be encoded
.roles(user.getRoles().toArray(new String[0])) .accountLocked(!user.isActive()) .build(); } }
// Wire it with PasswordEncoder @Bean public AuthenticationManager authenticationManager(
UserDetailsService userDetailsService, PasswordEncoder passwordEncoder) {
DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
provider.setUserDetailsService(userDetailsService);
provider.setPasswordEncoder(passwordEncoder); return new ProviderManager(provider); }
```

SecurityContext & ThreadLocal

The **SecurityContext** holds the current **Authentication** object. **SecurityContextHolder** uses a **ThreadLocal** strategy by default — each thread has its own **SecurityContext**. After the request is processed, **SecurityContextPersistenceFilter** clears the context. For reactive applications (WebFlux), **ReactiveSecurityContextHolder** uses **Reactor Context**.

```
// Accessing the current user anywhere in the application Authentication auth =
SecurityContextHolder.getContext().getAuthentication(); String username = auth.getName();
Collection<? extends GrantedAuthority> authorities = auth.getAuthorities(); // In a controller
– inject directly @GetMapping("/profile") public ResponseEntity<UserProfile> getProfile(
@AuthenticationPrincipal UserDetails userDetails) { return
ResponseEntity.ok(userService.getProfile(userDetails.getUsername())); } // For async
operations – propagate SecurityContext to child threads @Bean public TaskDecorator
securityContextTaskDecorator() { return runnable -> { SecurityContext context =
SecurityContextHolder.getContext(); return () -> { try {
SecurityContextHolder.setContext(context); runnable.run(); } finally {
SecurityContextHolder.clearContext(); } }; }; }
```

Method Security with @PreAuthorize

@PreAuthorize on method → Spring creates AOP proxy → intercepts call → SpEL expression evaluated

@PreAuthorize

Before method: hasRole, hasAuthority, #param

@PostAuthorize

After method: check return value

@PreFilter

Filter collection param before method

@PostFilter

Filter collection return value

```
// Enable method security @Configuration @EnableMethodSecurity // Spring Security 6 (replaces
@EnableGlobalMethodSecurity) public class MethodSecurityConfig {} // SpEL expressions in
@PreAuthorize @Service public class OrderService { @PreAuthorize("hasRole('ADMIN')") public
List<Order> getAllOrders() { ... } @PreAuthorize("hasAuthority('SCOPE_read:orders')") public
```

```
Order getOrder(Long id) { ... } // Access method parameter in SpEL @PreAuthorize("#userId == authentication.principal.id or hasRole('ADMIN')") public UserProfile getProfile(Long userId) { ... } // Check return value @PostAuthorize("returnObject.ownerId == authentication.principal.id") public Order getOrderById(Long orderId) { ... } // Custom expression with bean reference @PreAuthorize("@orderSecurity.canAccess(authentication, #orderId)") public Order getSecuredOrder(Long orderId) { ... } }
```

Authorization Architecture

Component	Role	Key Implementation
AccessDecisionManager	Makes access control decisions (legacy)	AffirmativeBased, ConsensusBased, UnanimousBased
AuthorizationManager	Modern replacement (Spring Security 5.0+)	AuthorityAuthorizationManager, RequestMatcherDelegatingAuthorizationManager
GrantedAuthority	Permission string held by Authentication	SimpleGrantedAuthority("ROLE_ADMIN"), "SCOPE_read:orders"
SecurityExpressionHandler	Evaluates SpEL security expressions	DefaultMethodSecurityExpressionHandler, DefaultWebSecurityExpressionHandler
PermissionEvaluator	Custom @PreAuthorize hasPermission() logic	DefaultPermissionEvaluator implements PermissionEvaluator interface

50+ Interview Questions & Answers

Q1. What is Spring Security?

A powerful, customizable authentication and authorization framework for Spring applications. Provides comprehensive security services: web request security, method security, authentication (form login, OAuth2, SAML, JWT), CSRF protection, session management.

Q2. What is DelegatingFilterProxy?

A standard javax.servlet.Filter registered in the servlet container (web.xml or programmatically). It bridges the servlet container lifecycle with Spring's ApplicationContext. Delegates to a Spring-managed bean (typically FilterChainProxy) by name.

Q3. What is FilterChainProxy?

The main Spring Security class that routes requests to SecurityFilterChain instances. Registered as a Spring bean (usually named 'springSecurityFilterChain'). Matches each request against chain patterns and delegates to the first matching chain.

Q4. What is SecurityFilterChain?

An interface mapping a request matcher to a list of security filters. Multiple SecurityFilterChain beans can exist — each handles different URL patterns (e.g., /api/** for JWT, rest for form login). Matched by @Order or request pattern priority.

Q5. What order do Spring Security filters run in?

Defined by FilterOrderRegistration. Key order: ChannelProcessingFilter, SecurityContextPersistenceFilter, ConcurrentSessionFilter, LogoutFilter, UsernamePasswordAuthenticationFilter, BearerTokenAuthenticationFilter, BasicAuthenticationFilter, ExceptionTranslationFilter, FilterSecurityInterceptor.

Q6. What is UsernamePasswordAuthenticationFilter?

Handles HTTP POST /login with username and password. Extracts credentials, creates UsernamePasswordAuthenticationToken, passes to AuthenticationManager. On success: stores Authentication in SecurityContext. On failure: calls failure handler.

Q7. What is AuthenticationManager?

Interface with single method: `authenticate(Authentication)`. The primary entry point for authentication. `ProviderManager` is the standard implementation — it delegates to a list of `AuthenticationProvider` instances.

Q8. What is ProviderManager?

Default `AuthenticationManager` that iterates through a list of `AuthenticationProvider` instances. Each provider can authenticate a specific `Authentication` type. If a provider supports the token, it tries to authenticate — first success wins.

Q9. What is DaoAuthenticationProvider?

`AuthenticationProvider` that uses `UserDetailsService` to load user details from a data store. Validates password with `PasswordEncoder`. Handles: account locked, disabled, expired, and bad credentials.

Q10. What is UserDetailsService?

Interface with `loadUserByUsername(String username)` returning `UserDetails`. The bridge between Spring Security and your user repository. Implement to load users from database, LDAP, etc. `DaoAuthenticationProvider` calls this during authentication.

Q11. What is UserDetails?

Interface representing a user in Spring Security. Contains: `getUsername()`, `getPassword()` (encoded), `getAuthorities()`, `isAccountNonExpired()`, `isAccountNonLocked()`, `isCredentialsNonExpired()`, `isEnabled()`. Implement or use `User` builder.

Q12. What is the SecurityContext?

Container for the `Authentication` object representing the current principal. Held in `SecurityContextHolder` (default: `ThreadLocal`). Populated at request start, cleared at request end by `SecurityContextPersistenceFilter`.

Q13. What is SecurityContextHolder?

Static utility class providing access to the `SecurityContext`. Default strategy: `ThreadLocalSecurityContextHolderStrategy` (`ThreadLocal`). Other strategies: `InheritableThreadLocal` (for child threads), `GlobalSecurityContextHolderStrategy` (for single-threaded apps like Swing).

Q14. What is Authentication?

Interface representing the principal. Contains: `getPrincipal()` (`UserDetails` or username), `getCredentials()` (password — cleared after auth), `getAuthorities()` (list of `GrantedAuthority`), `isAuthenticated()`. Common implementations: `UsernamePasswordAuthenticationToken`, `JwtAuthenticationToken`.

Q15. What is GrantedAuthority?

Interface for a permission or role: `getAuthority()` returns a `String`. Convention: roles prefixed with `ROLE_` (`ROLE_ADMIN`), OAuth2 scopes prefixed with `SCOPE_` (`SCOPE_read:orders`). `hasRole()` adds `ROLE_` prefix automatically; `hasAuthority()` does not.

Q16. What is the difference between hasRole and hasAuthority?

`hasRole('ADMIN')` checks for authority `'ROLE_ADMIN'` (adds prefix). `hasAuthority('ROLE_ADMIN')` checks for exact string `'ROLE_ADMIN'`. `hasAuthority('SCOPE_read:orders')` checks for exact string `'SCOPE_read:orders'`. Both use SpEL expressions.

Q17. What is @EnableWebSecurity?

Annotation enabling Spring Security's web security support. Imports `WebSecurityConfiguration`, `SpringWebMvcImportSelector`, etc. Required in Spring Boot when customizing security (though Spring Boot

auto-configures Security by default).

Q18. What is `@EnableMethodSecurity`?

Spring Security 6 annotation enabling method-level security. Enables: `@PreAuthorize`, `@PostAuthorize`, `@PreFilter`, `@PostFilter`. Replaces `@EnableGlobalMethodSecurity(prePostEnabled=true)`.

Q19. What is `@PreAuthorize`?

Method-level security annotation evaluated BEFORE the method executes. Uses SpEL: `hasRole()`, `hasAuthority()`, `hasAnyRole()`, `isAuthenticated()`, `isAnonymous()`, `#paramName` (method params), `authentication` (current auth), `@beanName` (custom logic).

Q20. What is `@PostAuthorize`?

Evaluated AFTER method executes — can access `returnObject` in SpEL. Used to check if the returned object belongs to the current user. Method still executes — only access to the result is controlled. Less common than `@PreAuthorize`.

Q21. How does AOP proxy work with method security?

`@EnableMethodSecurity` registers a `MethodInterceptor`. Spring wraps beans with `@PreAuthorize/@PostAuthorize` in a CGLIB or JDK dynamic proxy. When the method is called, the proxy intercepts, evaluates the SpEL expression, and either allows or throws `AccessDeniedException`.

Q22. What is `ExceptionHandlerFilter`?

Catches `AuthenticationException` (not authenticated) and `AccessDeniedException` (not authorized). For `AuthenticationException`: starts authentication entry point (e.g., redirect to login, return 401). For `AccessDeniedException`: anonymous user → start auth; authenticated user → 403.

Q23. What is `AuthenticationEntryPoint`?

Called when an unauthenticated user tries to access a secured resource. `LoginUrlAuthenticationEntryPoint`: redirects to login page. `HttpStatusEntryPoint`: returns 401. `BasicAuthenticationEntryPoint`: sends WWW-Authenticate header.

Q24. What is `AccessDeniedHandler`?

Called when an authenticated user lacks permission. Default: sends 403. `AccessDeniedHandlerImpl`: redirects to /403 page. Custom: implement `AccessDeniedHandler` to return custom error response.

Q25. What is `CsrfFilter`?

Validates CSRF tokens on state-changing requests (POST, PUT, DELETE, PATCH). Uses `CsrfTokenRepository` to generate and validate tokens. Default: `HttpSessionCsrfTokenRepository` (session) or `CookieCsrfTokenRepository` (SPA-friendly — stores in cookie).

Q26. When should you disable CSRF?

Only for stateless APIs (no session/cookies). If using JWT in Authorization header (not cookie), CSRF attacks are not possible. Disable with: `http.csrf(csrf -> csrf.disable())`. NEVER disable CSRF for session-based web apps.

Q27. What is `SessionManagementFilter`?

Handles session fixation protection, concurrent session control, and session creation policy. Configured via `sessionManagement()` in `HttpSecurity`.

Q28. What is session fixation attack?

Attacker sets a known session ID before the user logs in. After login, the user's session has the attacker's known ID — attacker hijacks. Prevention: change session ID after authentication (`migrateSession`,

newSession, changeSessionId — all supported by Spring).

Q29. What is concurrent session control?

Limits how many sessions a user can have simultaneously. Configure: `sessionManagement().maximumSessions(1).maxSessionsPreventsLogin(true)`. Requires `SessionRegistry`. Use Spring Session with Redis for distributed environments.

Q30. What is STATELESS session policy?

Tells Spring not to create or use HTTP sessions for security context. Required for REST APIs with JWT. `HttpSession` is never created. `SecurityContext` is loaded from the token on every request.

Q31. What is SecurityContextRepository?

Interface for saving and loading `SecurityContext` across requests. `HttpSessionSecurityContextRepository`: loads/saves from/to HTTP session (stateful). `NullSecurityContextRepository`: does nothing (stateless). `RequestAttributeSecurityContextRepository`: per-request only (Spring Security 5.7+).

Q32. What is the difference between authentication and authorization in Spring?

Authentication: verifying identity — who is the user (`AuthenticationManager`, `UserDetailsService`). Authorization: verifying permission — what can the user do (`AuthorizationManager`, `@PreAuthorize`, `HttpSecurity.authorizeHttpRequests`).

Q33. What is PasswordEncoder?

Interface for encoding and verifying passwords. `encode(rawPassword)` for storage. `matches(rawPassword, encodedPassword)` for verification. Never implement your own — use `BCryptPasswordEncoder`, `Pbkdf2PasswordEncoder`, or `DelegatingPasswordEncoder`.

Q34. What is DelegatingPasswordEncoder?

A `PasswordEncoder` that supports multiple encoding formats simultaneously. Format: `{id}encodedPassword`. Allows migration: encode with `BCrypt`, can still verify old `SCrypt`/`PBKDF2` hashes. Created via `PasswordEncoderFactories.createDelegatingPasswordEncoder()`.

Q35. What is the PasswordUpgradeManager?

When a user logs in with an old password encoding, `DelegatingPasswordEncoder` detects the mismatch. `UserDetailsService.updatePassword()` is called with the newly encoded password. Transparent migration without requiring password reset.

Q36. How do you add a custom filter to Spring Security?

`http.addFilterBefore(new MyFilter(), UsernamePasswordAuthenticationFilter.class)` or `addFilterAfter`, `addFilterAt`. Custom filter must extend `OncePerRequestFilter` or implement `Filter`. Can inject beans via constructor injection.

Q37. What is OncePerRequestFilter?

Spring's abstract `Filter` base class guaranteeing the filter executes only once per request (prevents double-invocation in forward/include scenarios). Override `doFilterInternal(request, response, filterChain)`.

Q38. How does Spring Security handle Remember-Me?

`RememberMeAuthenticationFilter` reads a remember-me cookie. Two implementations: `SimplePersistentTokenRepository` (insecure — encodes username+expiry in cookie), `PersistentTokenRepository` (stores random token in DB — recommended). Configure: `http.rememberMe(rm -> rm.tokenRepository(jdbcTokenRepository))`.

Q39. What is AuthenticationSuccessHandler?

Called after successful authentication. `SavedRequestAwareAuthenticationSuccessHandler`: redirects to the originally requested URL (default). Custom implementations: update last login time, log audit event, redirect based on role.

Q40. What is `AuthenticationFailureHandler`?

Called after failed authentication. `SimpleUrlAuthenticationFailureHandler`: redirects to `/login?error`. Custom: limit login attempts, lock account, return JSON error for API.

Q41. What is `LogoutHandler` vs `LogoutSuccessHandler`?

`LogoutHandler`: performs cleanup during logout (multiple handlers chained) — clear session, clear remember-me, revoke tokens. `LogoutSuccessHandler`: determines what to do after logout (single handler) — redirect to login page or IdP logout URL.

Q42. What is `SecurityExpressionRoot`?

Provides the built-in SpEL functions: `hasRole()`, `hasAuthority()`, `isAuthenticated()`, `isAnonymous()`, `permitAll()`, `denyAll()`, `isFullyAuthenticated()`, `isRememberMe()`, `principal`, `authentication`. Extended by method and web security expression roots.

Q43. How do you create custom SpEL security expressions?

Create a class extending `SecurityExpressionRoot` or `MethodSecurityExpressionRoot`. Add custom methods. Register via `MethodSecurityExpressionHandler.setExpressionParser()` or `DefaultMethodSecurityExpressionHandler` subclass. Reference as: `@PreAuthorize('@myService.check(#id)').`

Q44. What is the `AnonymousAuthenticationFilter`?

Fills the `SecurityContext` with an anonymous `Authentication` object (`AnonymousAuthenticationToken`) when no other authentication is present. Allows expressions like `isAnonymous()` and `denyAll()` to work correctly. Always near the end of the filter chain.

Q45. What is `GrantedAuthorityDefaults`?

Allows customizing the role prefix (default: `ROLE_`). Configure: `@Bean GrantedAuthorityDefaults grantedAuthorityDefaults() { return new GrantedAuthorityDefaults(""); } — removes ROLE_ prefix, so hasRole('ADMIN') checks for 'ADMIN' not 'ROLE_ADMIN'.`

Q46. What is `NullSecurityContextRepository`?

Tells Spring Security not to load or save the `SecurityContext` to/from the session. Used with stateless APIs. Configured via: `http.securityContext(sc) -> sc.securityContextRepository(new NullSecurityContextRepository())`.

Q47. What is `WebSecurityCustomizer`?

Spring Security 5.4+ way to configure `WebSecurity` directly (replaces `WebSecurityConfigurerAdapter.configure(WebSecurity)`). Used to ignore certain paths completely (no filters): `web.ignoring().requestMatchers('/actuator/health', '/static/**')`.

Q48. What is `SecurityMatcher`?

Restricts which requests a `SecurityFilterChain` applies to. `http.securityMatcher('/api/**')` — chain only processes requests matching this pattern. Unlike `HttpSecurity.authorizeHttpRequests`, `securityMatcher` determines if the chain applies at all.

Q49. What is the difference between `permitAll()` and `ignoring()`?

`permitAll()`: request goes through all filters (CSRF, session management, etc.) but no authorization check. `ignoring()`: request bypasses ALL Spring Security filters entirely — faster, but no security processing at all.

Use `permitAll()` for login/static pages; `ignoring()` only for truly public static resources.

Q50. How does Spring Security work with Spring Boot?

Spring Boot auto-configures Spring Security via `SecurityAutoConfiguration` and `UserDetailsServiceAutoConfiguration`. Provides default `SecurityFilterChain` with form login, basic auth, random password. Override by defining your own `SecurityFilterChain` bean(s).

Q51. What is `@AuthenticationPrincipal`?

Spring MVC argument resolver annotation injecting the current principal into controller methods. `@AuthenticationPrincipal UserDetails userDetails` — injects the `UserDetails` object. `@AuthenticationPrincipal(expression='email')` `String email` — extracts a property via SpEL.

Q52. How do you test Spring Security?

`@WithMockUser` — simulates authenticated user. `@WithUserDetails` — loads `UserDetails` from `UserDetailsService`. `@WithAnonymousUser` — simulates anonymous. `MockMvc: .with(SecurityMockMvcRequestPostProcessors.user('user').roles('ADMIN'))`. For JWT: `.with(jwt().authorities(new SimpleGrantedAuthority('SCOPE_read')))`.